UNIVERSITY OF OSLO

A Self-Adaptive Digital Twin Architecture for Dynamic Resource Management

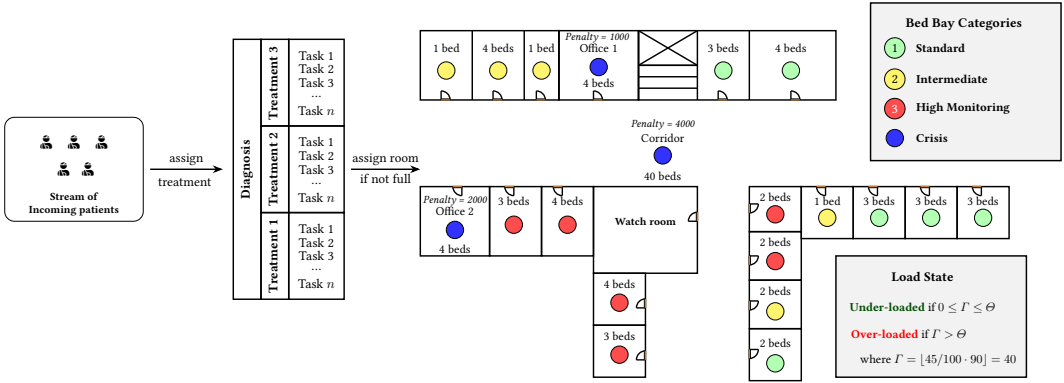
Riccardo Sieve, Paul Kobialka, Andrea Pferscher, Nelly Bencomo, Silvia Lizeth Tapia Tarifa, Buster Salomon Rasmussen, Einar Broch Johnsen

SEAMS 2026

April 13, 2026

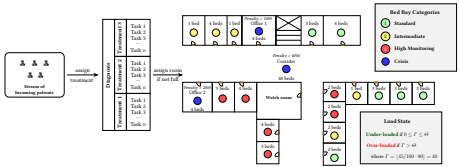
Contents

Why do we need it?

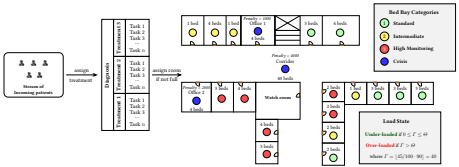


Why do we need it?

- Hospital demand changes quickly; resource capacity does not

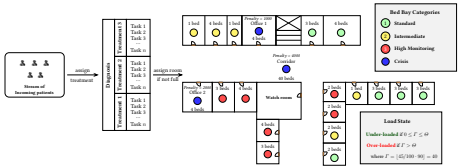


Why do we need it?



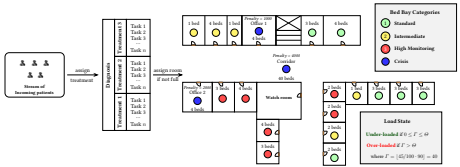
- Hospital demand changes quickly; resource capacity does not
- Admission is often handled manually

Why do we need it?



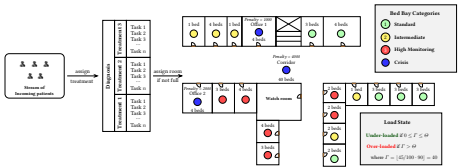
- Hospital demand changes quickly; resource capacity does not
- Admission is often handled manually
- Depends on several factors

Why do we need it?



- Hospital demand changes quickly; resource capacity does not
- Admission is often handled manually
- Depends on several factors
- Static architectures struggle when the underlying structure changes

Why do we need it?



- Hospital demand changes quickly; resource capacity does not
- Admission is often handled manually
- Depends on several factors
- Static architectures struggle when the underlying structure changes
- We need runtime adaptation with trustworthy model alignment

Baseline: BedreFlyt and the gap

- BedreFlyt combines semantic modelling, forecasting, and optimization

Baseline: BedreFlyt and the gap

- BedreFlyt combines semantic modelling, forecasting, and optimization
- It produces bed-bay recommendations for human operators

Baseline: BedreFlyt and the gap

- BedreFlyt combines semantic modelling, forecasting, and optimization
- It produces bed-bay recommendations for human operators
- Human-in-the-loop actions can diverge from DT suggestions

Baseline: BedreFlyt and the gap

- BedreFlyt combines semantic modelling, forecasting, and optimization
- It produces bed-bay recommendations for human operators
- Human-in-the-loop actions can diverge from DT suggestions
- Main gap: the original ward layout is treated as fixed

What is the goal

- A self-adaptive architecture for resource management (BedreFlyt use case) that combines

What is the goal

- A self-adaptive architecture for resource management (BedreFlyt use case) that combines
  - Evolving semantic model to represent changing resources at runtime

What is the goal

- A self-adaptive architecture for resource management (BedreFlyt use case) that combines
  - Evolving semantic model to represent changing resources at runtime
  - Lifecycle manager to monitor context and trigger adaptation

What is the goal

- A self-adaptive architecture for resource management (BedreFlyt use case) that combines
  - Evolving semantic model to represent changing resources at runtime
  - Lifecycle manager to monitor context and trigger adaptation
  - Penalty-based optimisation for cost-aware reconfiguration

What is the goal

- A self-adaptive architecture for resource management (BedreFlyt use case) that combines
  - Evolving semantic model to represent changing resources at runtime
  - Lifecycle manager to monitor context and trigger adaptation
  - Penalty-based optimisation for cost-aware reconfiguration
- Human-in-the-loop supervision over the adaptation loop

Main contributions

- **Self-adaptive DT architecture:** integrates semantic reflection and runtime orchestration for dynamic ward structure changes

Main contributions

- **Self-adaptive DT architecture:** integrates semantic reflection and runtime orchestration for dynamic ward structure changes
- **Penalty-based optimisation model:** supports adaptation decisions by balancing feasibility and adaptation cost

Main contributions

- **Self-adaptive DT architecture:** integrates semantic reflection and runtime orchestration for dynamic ward structure changes
- **Penalty-based optimisation model:** supports adaptation decisions by balancing feasibility and adaptation cost
- **Formalised adaptation objective:** ensure patient allocation feasibility with the fewest rooms while minimising penalty

Main contributions

- **Self-adaptive DT architecture:** integrates semantic reflection and runtime orchestration for dynamic ward structure changes
- **Penalty-based optimisation model:** supports adaptation decisions by balancing feasibility and adaptation cost
- **Formalised adaptation objective:** ensure patient allocation feasibility with the fewest rooms while minimising penalty
- **Human-supervised autonomy:** operators approve actions while the DT keeps structural and behavioural models aligned

Contents

What is a Digital Twin

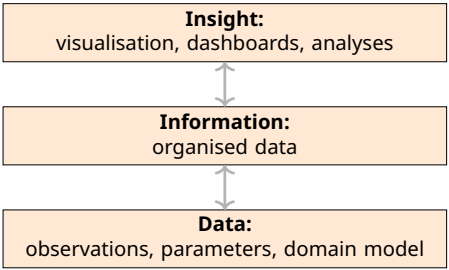
Digital twins are, as defined by NASEM

NASEM Definition of DT (2024)

A digital twin is a set of virtual information constructs that mimics the structure, context, and behaviour of a natural, engineered, or **social system** (or system-of-systems), is dynamically updated with data from its physical twin, has a predictive capability, and **informs decisions** that realise value. The bidirectional interaction between the virtual and the physical is central to the digital twin[a].

[a] National Academies of Sciences, Engineering, and Medicine (NASEM) (2024): Foundational Research Gaps and Future Directions for Digital Twins. The National Academies Press

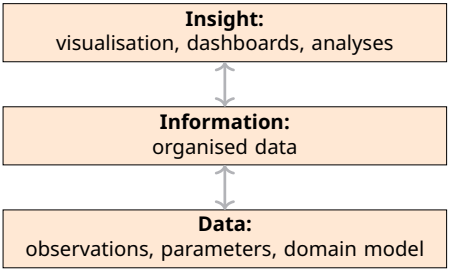
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")

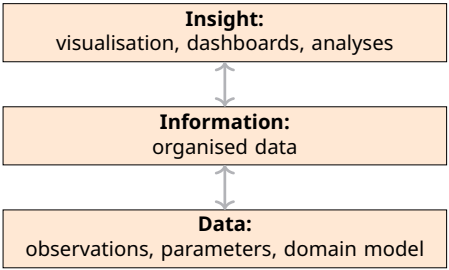
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if / what should”)

Digital Twins components

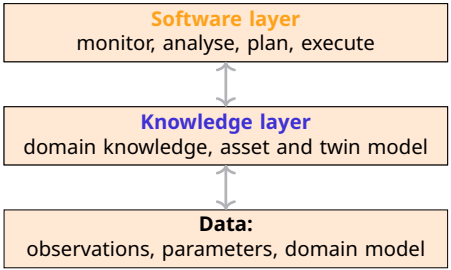
Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

Digital Twins components



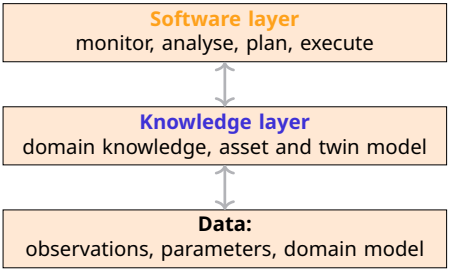
Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

Digital Twins components



Capabilities for different insights:

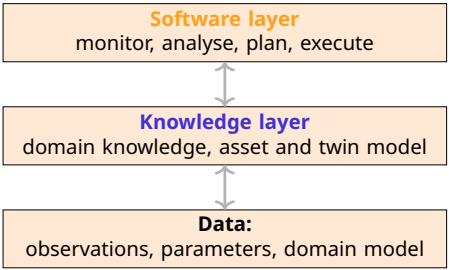
- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

- DT must adapt both structurally and behaviourally

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

- DT must adapt both structurally and behaviourally
- States of the DT cycles over time and they need to be captured

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

smolang.org

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

SMOL: Semantic Model Object Language

- Smalll OO programming system
- Ontology reasoners allow querying the KB
- Used to create the digital model

smolang.org

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

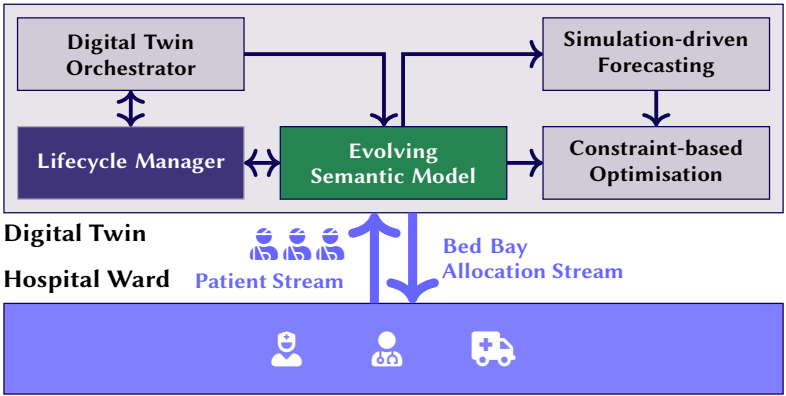
smolang.org

behavioural twins in SMOL

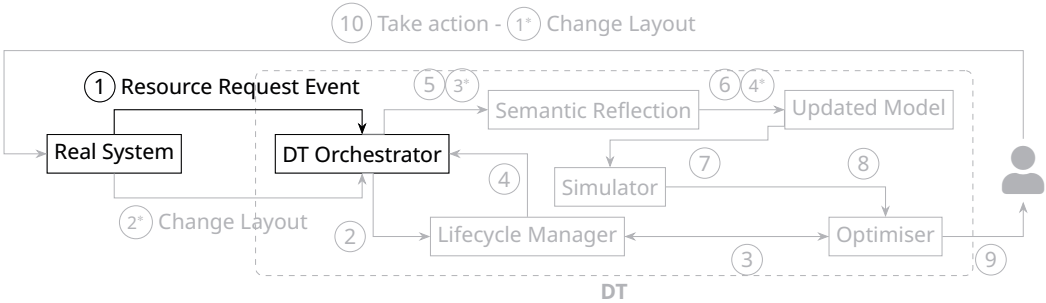
- SMOL can encapsulate (simulation) models based on the FMI standard
- Can automatically adapt the object states in the KB at runtime through semantic lifting

Contents

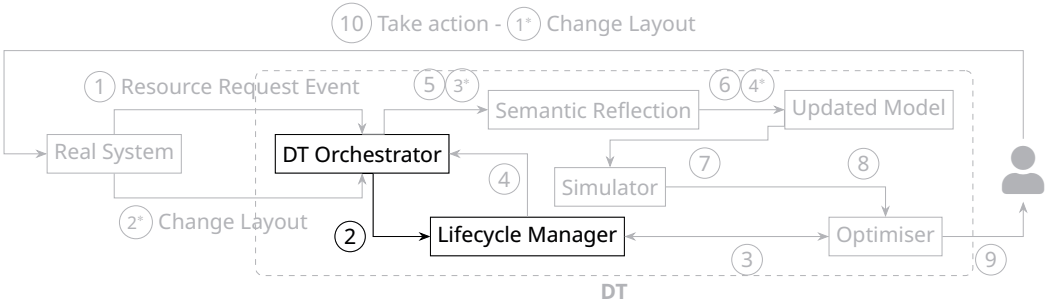
Architecture



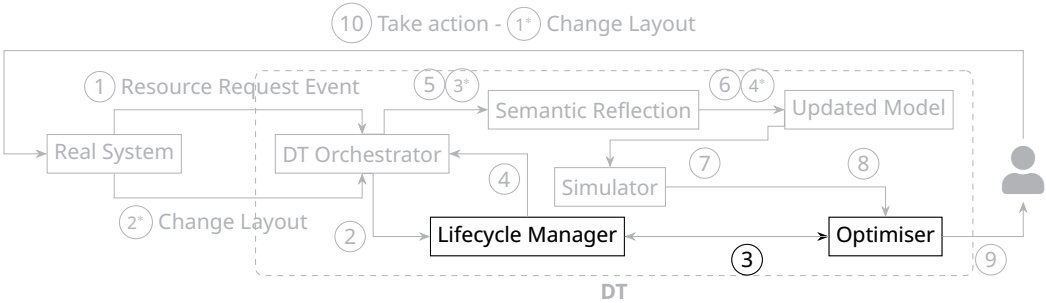
Human-in-the-loop



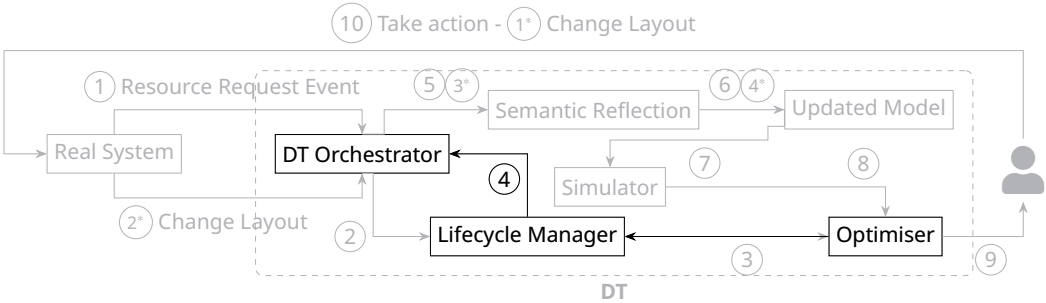
Human-in-the-loop



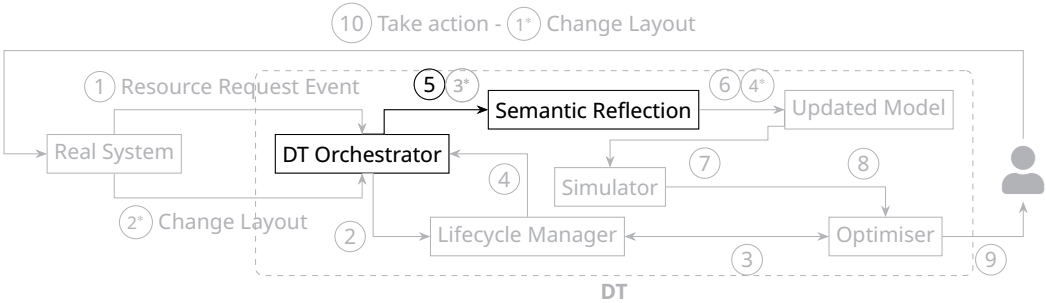
Human-in-the-loop



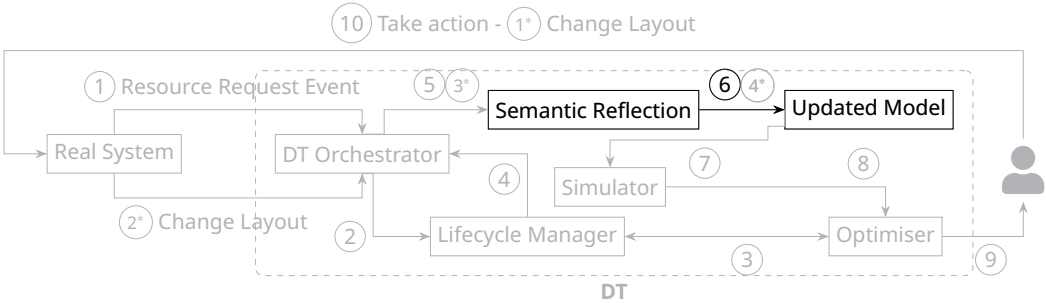
Human-in-the-loop



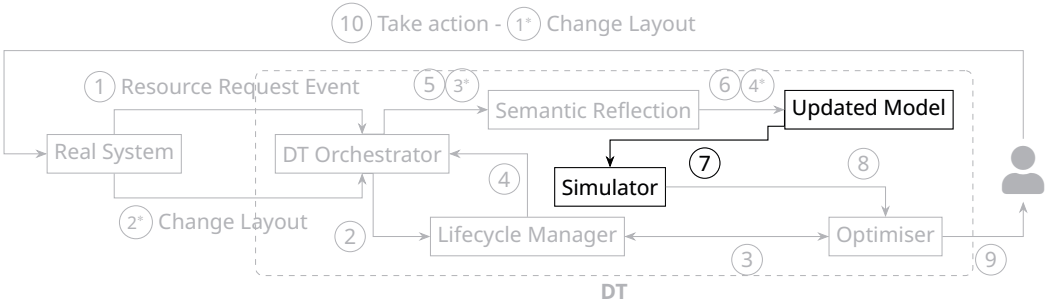
Human-in-the-loop



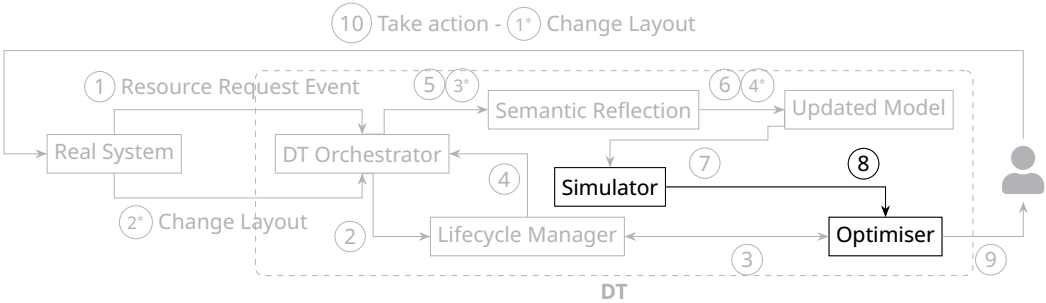
Human-in-the-loop



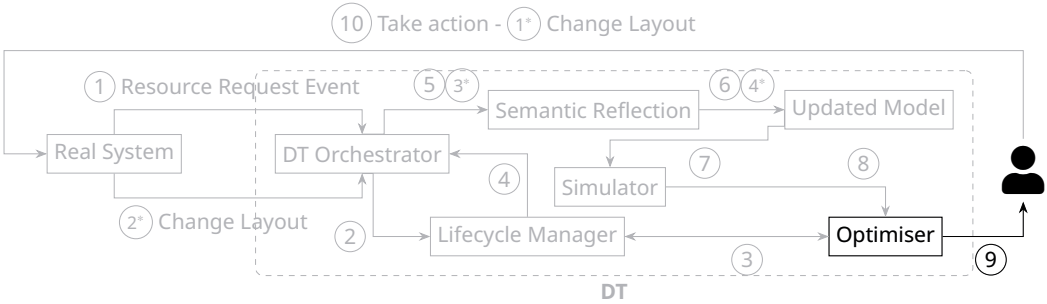
Human-in-the-loop



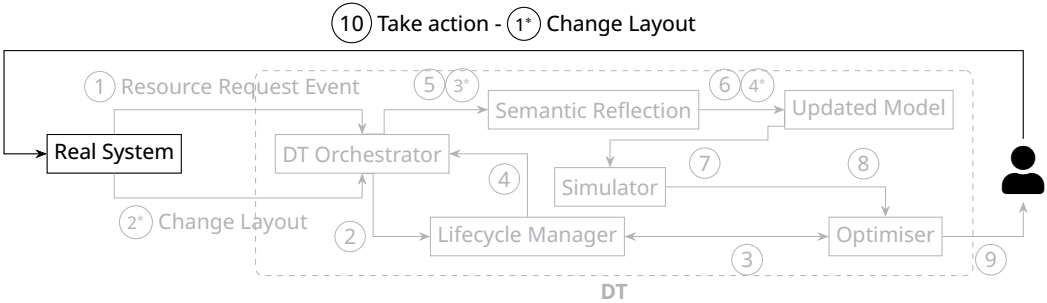
Human-in-the-loop



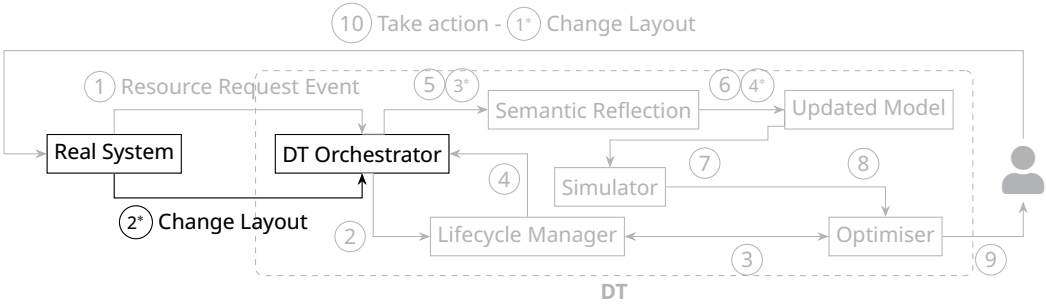
Human-in-the-loop



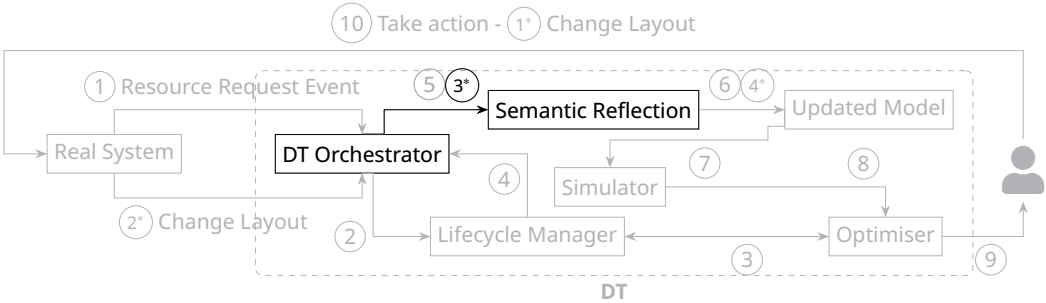
Human-in-the-loop



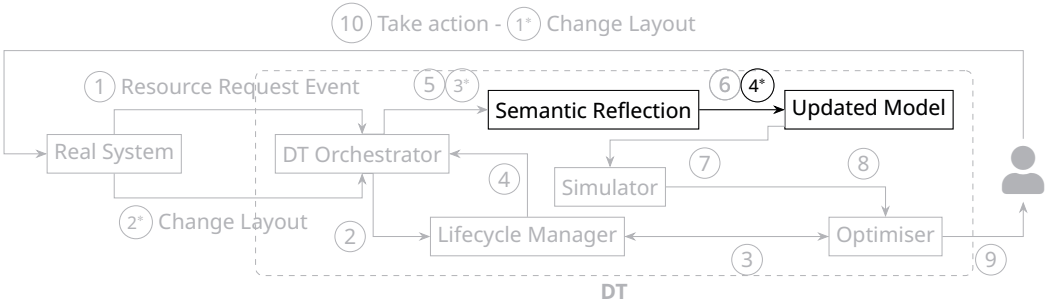
Human-in-the-loop



Human-in-the-loop



Human-in-the-loop



Human-in-the-loop (cont.)

- The human-in-the-loop remains the final authority for adaptation actions

Human-in-the-loop (cont.)

- The human-in-the-loop remains the final authority for adaptation actions
- The DT proposes either *allocation updates* or *layout reconfiguration*

Human-in-the-loop (cont.)

- The human-in-the-loop remains the final authority for adaptation actions
- The DT proposes either *allocation updates* or *layout reconfiguration*
- Once approved, the lifecycle manager executes the selected adaptation path

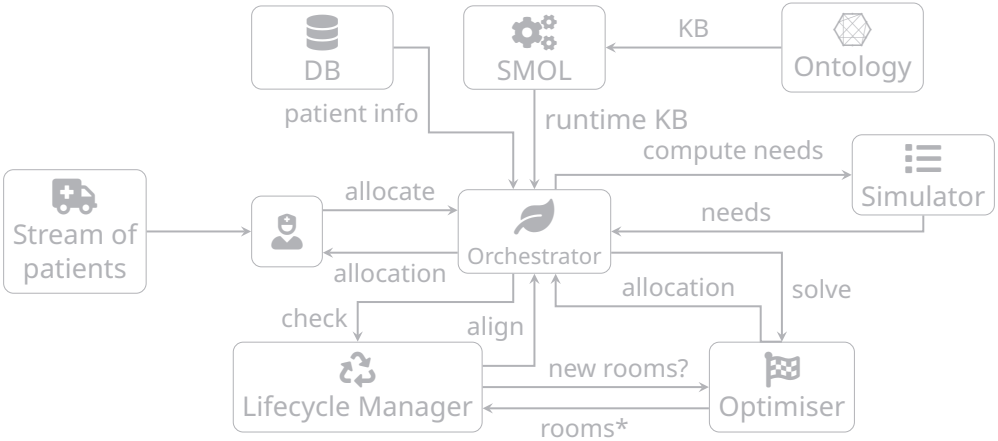
Human-in-the-loop (cont.)

- The human-in-the-loop remains the final authority for adaptation actions
- The DT proposes either *allocation updates* or *layout reconfiguration*
- Once approved, the lifecycle manager executes the selected adaptation path
- Semantic reflection re-aligns the runtime model after structural changes

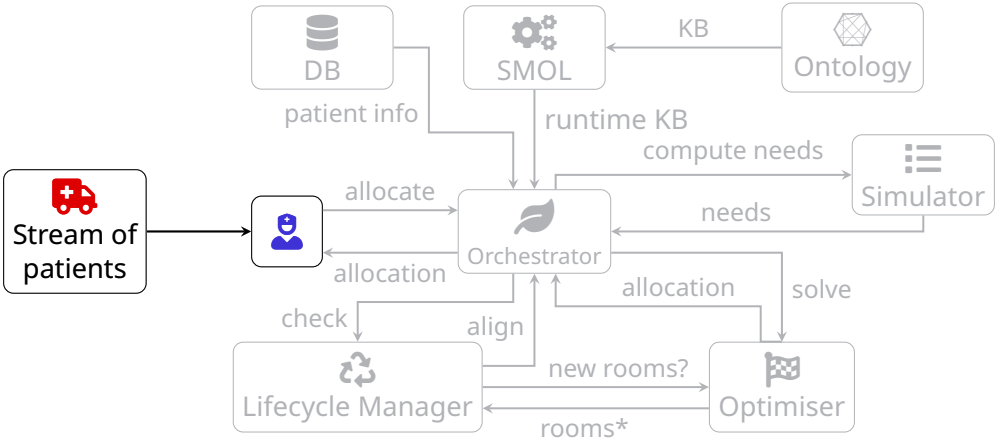
Human-in-the-loop (cont.)

- The human-in-the-loop remains the final authority for adaptation actions
- The DT proposes either *allocation updates* or *layout reconfiguration*
- Once approved, the lifecycle manager executes the selected adaptation path
- Semantic reflection re-aligns the runtime model after structural changes
- This enables supervised autonomy: controlled self-adaptation instead of static decision support

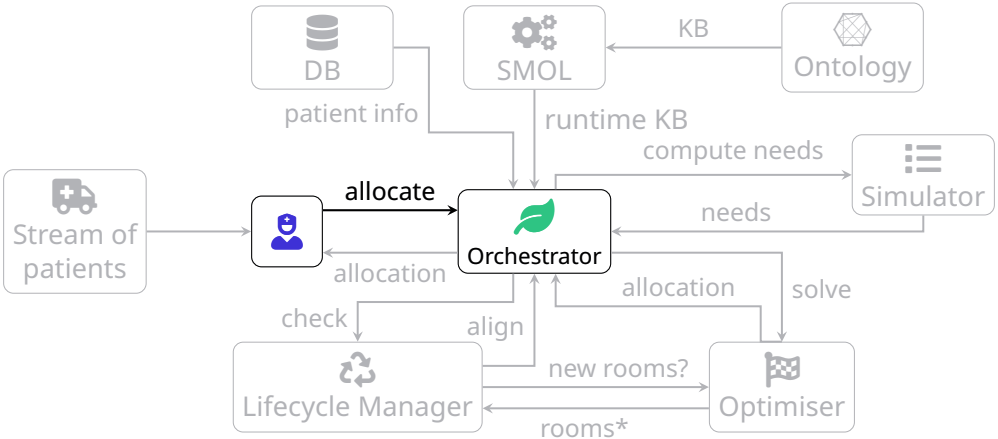
Allocation of patients



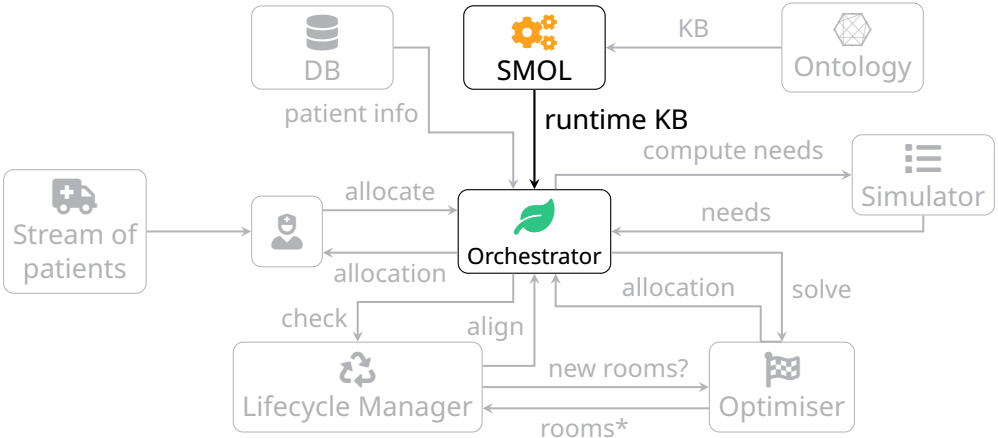
Allocation of patients



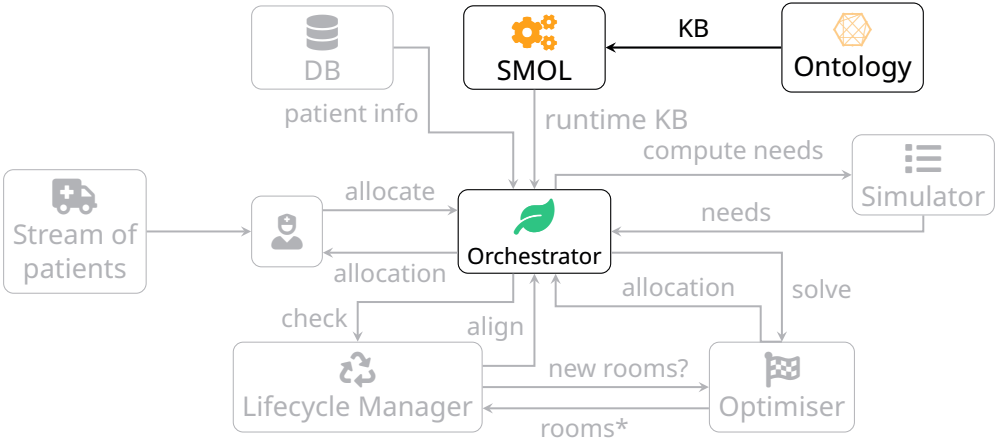
Allocation of patients



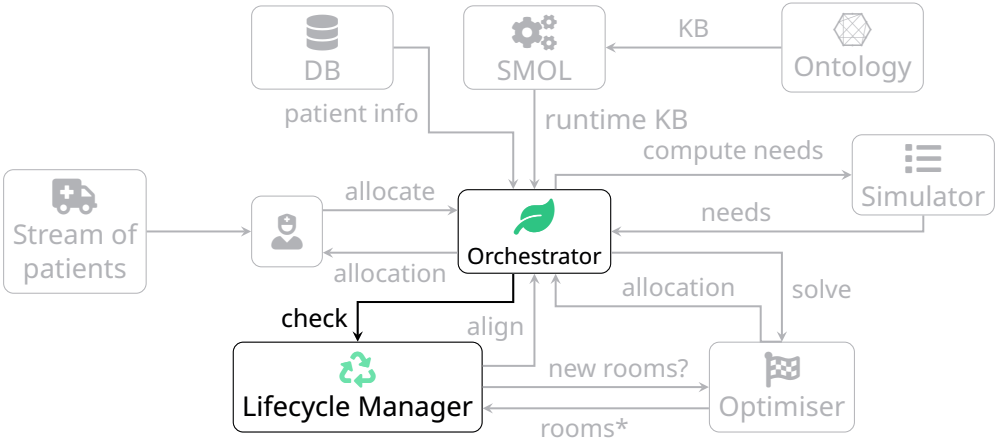
Allocation of patients



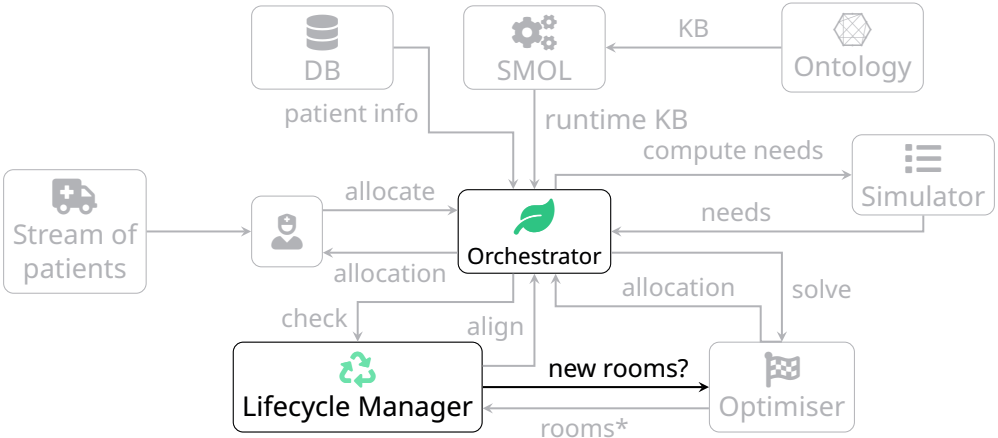
Allocation of patients



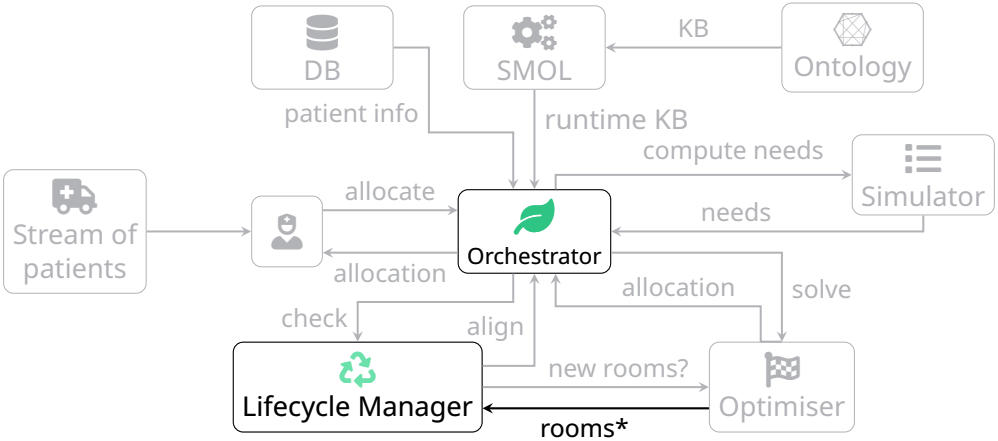
Allocation of patients



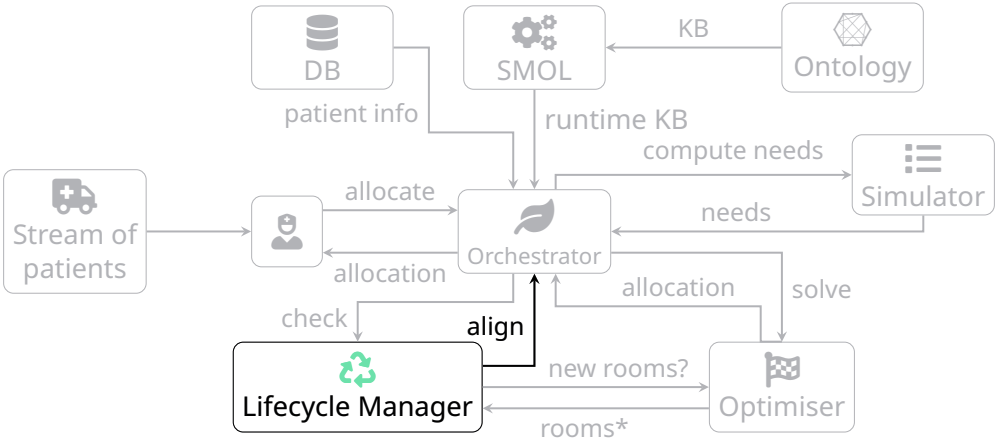
Allocation of patients



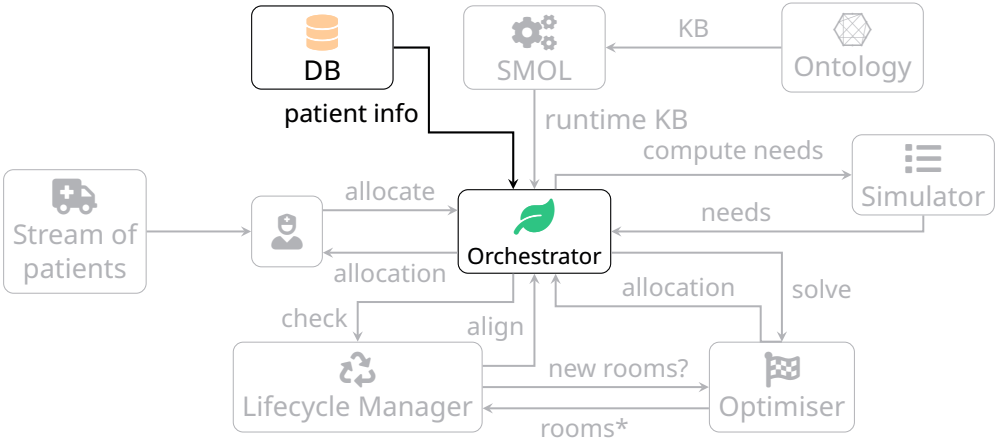
Allocation of patients



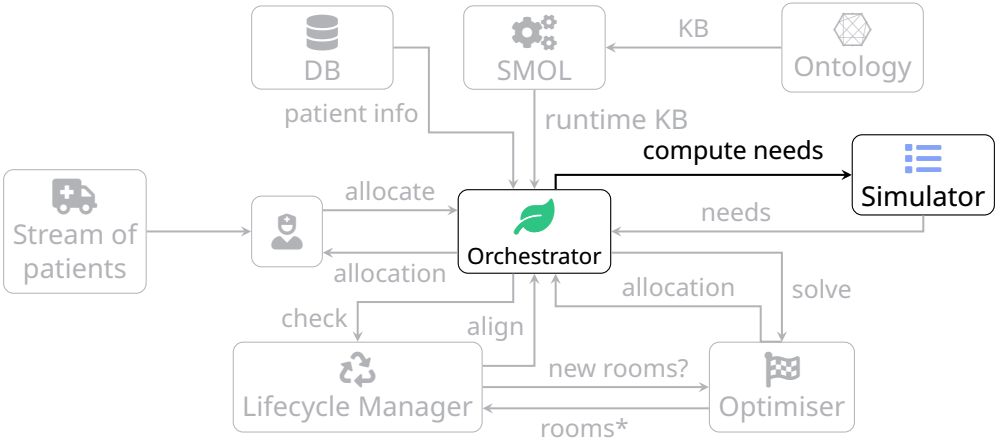
Allocation of patients



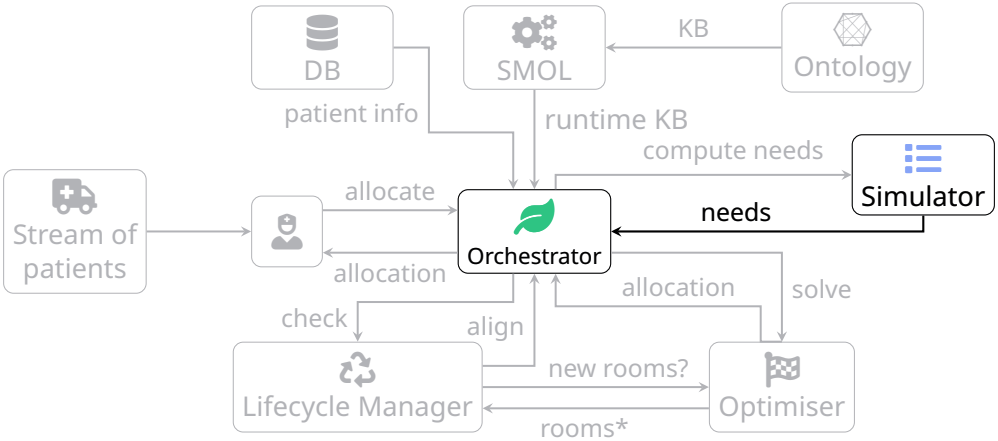
Allocation of patients



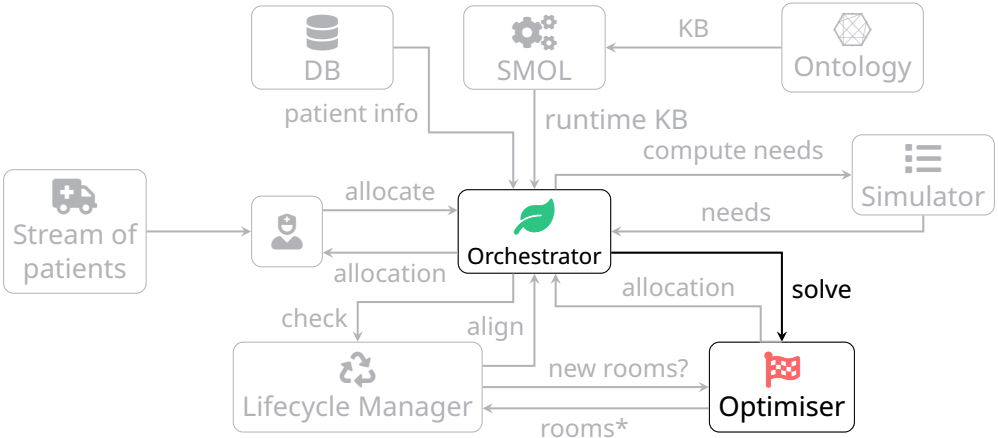
Allocation of patients



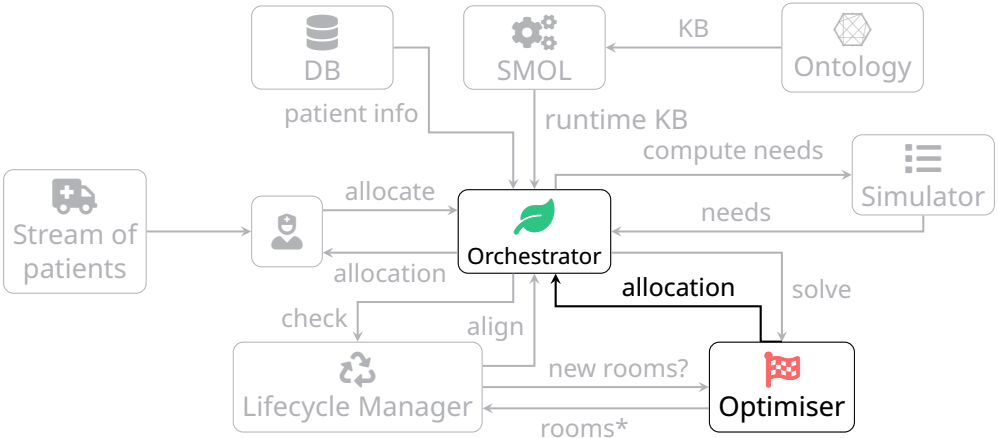
Allocation of patients



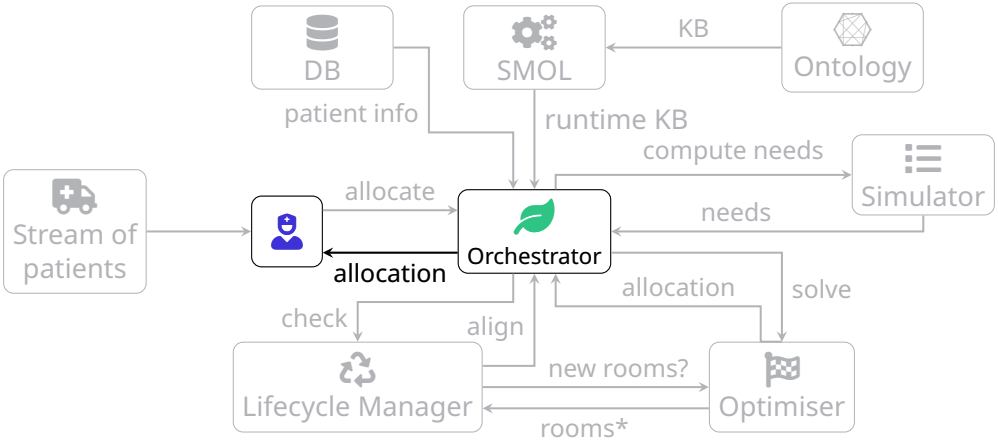
Allocation of patients



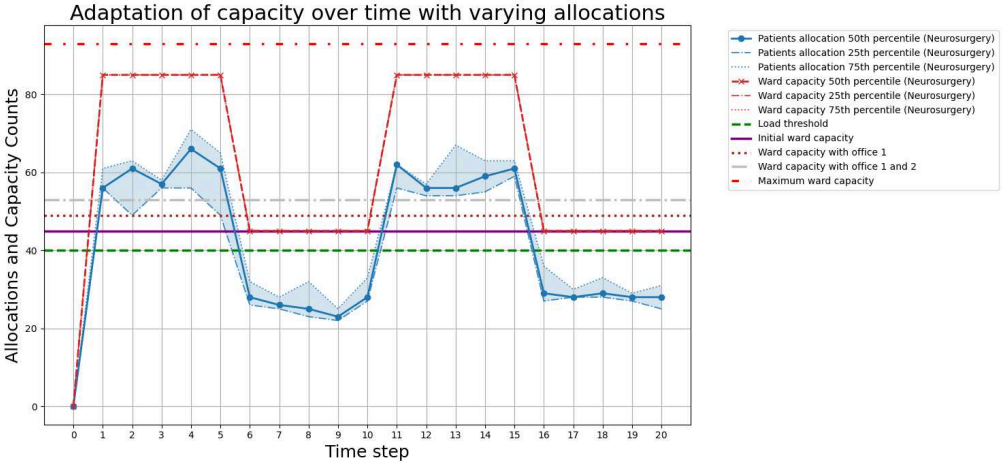
Allocation of patients



Allocation of patients



Allocation of patients (cont.)



Takeaways and Limits

- The architecture enables principled runtime resource adaptation

Takeaways and Limits

- The architecture enables principled runtime resource adaptation
- Key trade-off: correctness and alignment versus runtime cost

Takeaways and Limits

- The architecture enables principled runtime resource adaptation
- Key trade-off: correctness and alignment versus runtime cost
- SMOL allows adaptation through semantic lifting, but it comes as a cost

Takeaways and Limits

- The architecture enables principled runtime resource adaptation
- Key trade-off: correctness and alignment versus runtime cost
- SMOL allows adaptation through semantic lifting, but it comes as a cost
- Stable demand reduces reflection overhead as the semantic lifting is less likely to be triggered

Takeaways and Limits

- The architecture enables principled runtime resource adaptation
- Key trade-off: correctness and alignment versus runtime cost
- SMOL allows adaptation through semantic lifting, but it comes as a cost
- Stable demand reduces reflection overhead as the semantic lifting is less likely to be triggered
- Penalty-guided optimisation turns the resilience-versus-cost relation into an explicit reasoning element of the architecture

Takeaways and Limits

- The architecture enables principled runtime resource adaptation
- Key trade-off: correctness and alignment versus runtime cost
- SMOL allows adaptation through semantic lifting, but it comes as a cost
- Stable demand reduces reflection overhead as the semantic lifting is less likely to be triggered
- Penalty-guided optimisation turns the resilience-versus-cost relation into an explicit reasoning element of the architecture
- Current limits: manually tuned penalties and threshold

Future directions

- Extend the work to enable self-adaptive techniques for different wards

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Include different components to simulate (e.g. staff management)

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital
- Explore different strategies based on uncertainties of non-adaptive approach

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital
- Explore different strategies based on uncertainties of non-adaptive approach
- Currently, SMOL provides overhead in the execution, leading to a trade-off between correctness and performance. We plan on investigate and improve the overall performance